

SHNOOR MEET — Meeting & Collaboration Platform

Project Documentation

Developed By: SHNOOR INTERNATIONAL LLC

Technology: React.js | Vite | Node.js | Express.js | PostgreSQL | Socket.IO | Python

Abstract

This report documents **Shnoor Meet** — a browser-based meeting and collaboration platform developed to provide secure, fast, and intelligent virtual communication for modern teams and organizations. The system combines secure user authentication, instant room creation, real-time video conferencing, in-meeting chat, host moderation controls, and a custom trainable AI assistant to enhance meeting productivity.

The application is built using a modern multi-layer architecture consisting of a **React.js + Vite frontend** for responsive user interaction, a **Node.js + Express.js backend** for API processing and business logic, a **PostgreSQL relational database** for structured data storage, **Socket.IO signaling** for real-time communication, and a **Python-based chatbot engine** trained from local CSV datasets instead of external AI APIs.

The primary goal of this project is to solve practical collaboration challenges by delivering a lightweight and scalable meeting platform that supports real-time communication, room-level management, and intelligent contextual assistance without depending on third-party chatbot providers. The platform enables teams to collaborate efficiently while maintaining flexibility and control over data and system behavior.

1. Introduction

PROJECT PURPOSE

The purpose of Shnoor Meet is to provide an integrated video-conferencing solution with built-in meeting assistance. Unlike a minimal room-sharing demo, the codebase combines authentication, persistent meeting records, collaboration controls, and a trainable chatbot layer that can answer domain-specific questions and execute selected actions.

➤ Business Problem Solved

- Teams need a simple web interface to create and join meetings quickly.
- Meeting platforms often require users to navigate multiple controls manually during a live session.
- Generic AI tools can be expensive, opaque, or unsuitable for domain-specific support workflows.
- Realtime collaboration systems need signaling, presence tracking, and host-level moderation to be practical.

➤ Target Users

- Registered users who want persistent identity and room creation capability.
- Guest participants who join active rooms without a permanent account.
- Meeting hosts who require room moderation tools.
- Future developers and maintainers who need a readable modular codebase.

➤ Key Value Delivered

Shnoor Meet provides value through an end-to-end collaboration workflow: account creation, secure login, one-click room creation, WebRTC-based participant communication, in-room messaging, host governance, and a self-trained chatbot that can guide users, perform selected actions, learn from feedback, and answer image-linked questions without external LLM services.

2. Objectives

- Build a multi-user web meeting platform with secure authentication and room persistence.
- Provide realtime collaboration through Socket.IO signaling and browser-native WebRTC media exchange.
- Enable meeting operations such as create, join, leave, chat, screen share, reactions, and hand raise.
- Provide host controls to manage participant capabilities for microphone, video, reactions, and screen sharing.
- Develop a local, self-trained chatbot using CSV-based training data and no external conversational API dependency.
- Support user feedback loops so incorrect chatbot answers can be corrected and retrained into the model.
- Add image-aware chatbot capability for screenshot-based or image-based queries.
- Allow the assistant to inspect live meeting state from the current room screen for context-specific replies.

3. Technology Stack

Frontend

Technology	Role In Project	Evidence From Code
React 19	UI rendering, page composition, component state, hooks	<code>frontend/src/App.jsx</code> , page and component modules
Vite	Frontend dev server and production build tooling	<code>frontend/package.json</code> , <code>vite.config.js</code>

React Router DOM	Client-side routing for landing, login, signup, and meeting room pages	<code>App.jsx</code>
Socket.IO Client	Realtime connection from meeting page to backend signaling server	<code>MeetingRoom.jsx</code>
Lucide React	UI iconography for auth, landing, meeting controls, and chatbot controls	Imported across page/component modules
Emoji Picker React	Emoji reaction picker inside live meeting room	<code>MeetingRoom.jsx</code>
Pure CSS / custom variables	Glassmorphism styling, control bars, responsive sections, animation	<code>index.css</code> , page CSS files

Backend

Technology	Role In Project	Evidence From Code
Node.js	Primary server runtime for HTTP APIs and realtime services	<code>backend/server.js</code>
Express 5	REST API routing, middleware registration, JSON handling	<code>server.js</code> , route modules
Socket.IO	WebRTC signaling, room events, chat, reactions, host control updates	<code>server.js</code>
Python HTTP server	Custom chatbot runtime with training, inference, and image-learning endpoints	<code>backend/chatbot/service.py</code>

Database

Technology	Role In Project	Evidence From Code
PostgreSQL	Persistent storage for users, meetings, participants, and chatbot feedback events	<code>init.sql</code> , <code>db.js</code>
pg	Connection pooling and query execution from Node.js	<code>db.js</code>

Authentication

Technology	Role In Project	Evidence From Code
bcryptjs	Password hashing during signup and chatbot-triggered user creation	<code>routes/auth.js</code> , <code>server.js</code>
Json web token	JWT issuance on signup/login	<code>routes/auth.js</code>
LocalStorage-based session persistence	Stores user object and token on the client	<code>AuthContext.jsx</code>

4. System Architecture

High-Level Architecture

ARCHITECTURE SUMMARY

The solution follows a **client-server-service** model. The React frontend is responsible for routing, meeting UI, chatbot UI, and client-side meeting state. The Node/Express backend provides HTTP APIs, realtime Socket.IO signaling, PostgreSQL connectivity, and orchestration for the Python chatbot process. The Python service performs chatbot training and inference separately, while still being controlled by the Node application.

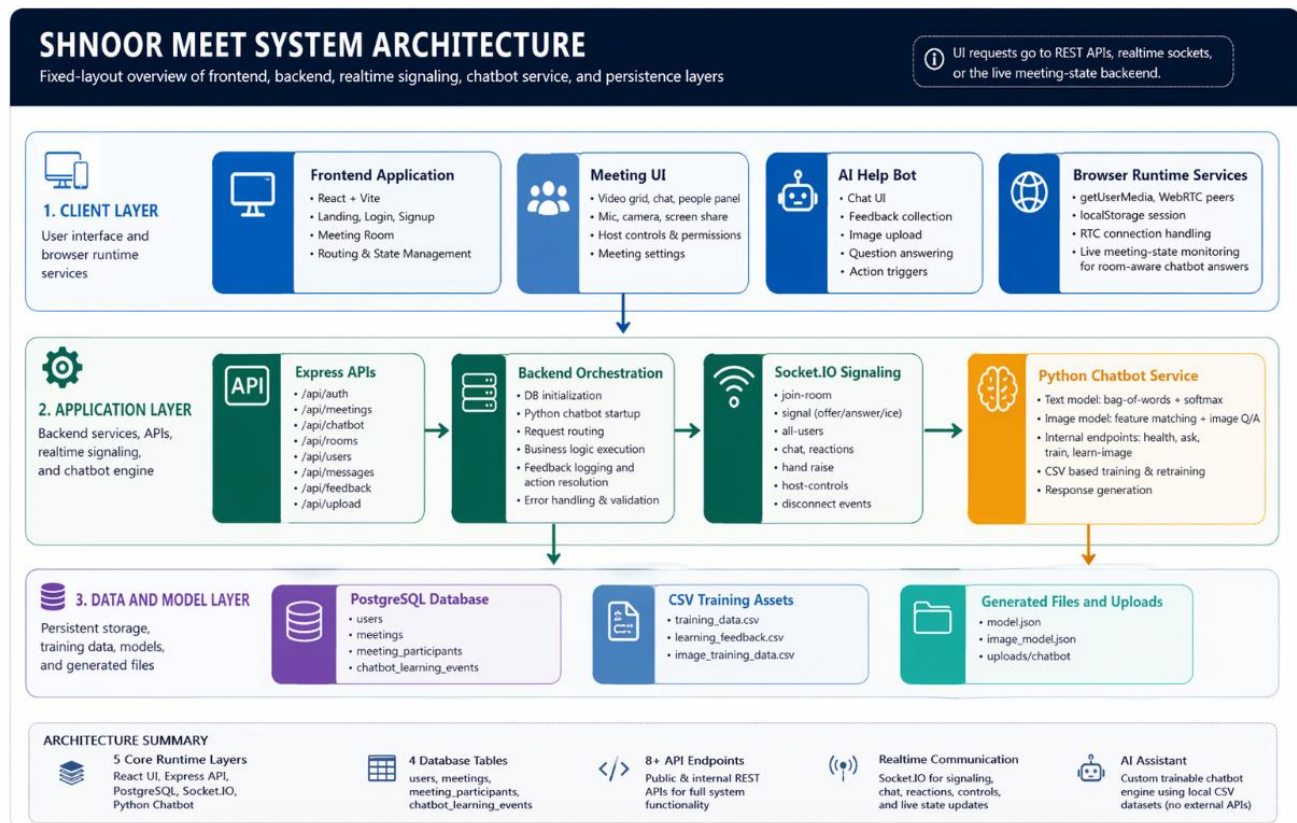


Fig : System Architecture of Shnoor Meet

Primary Components

- React frontend:** pages, controls, chatbot UI, room state, media access, socket client.
- Express backend:** auth routes, meeting routes, chatbot proxy endpoints, image upload endpoint.
- Socket.IO layer:** room join, signaling, chat, reactions, host settings propagation.
- PostgreSQL:** users, meetings, meeting participants schema foundation, feedback event logging.
- Python chatbot service:** local training pipeline, intent classification, image learning.

Data Flow

- User actions start in the browser and either hit REST APIs or realtime socket channels.
- Meeting room operations exchange signaling data over Socket.IO while audio/video travels peer-to-peer over WebRTC.
- Chatbot requests first attempt a live meeting-state answer in the browser, then fall back to backend inference.

- Feedback corrections write to PostgreSQL and CSV training files, then retrigger chatbot retraining.

Request-Response Lifecycle

1. Frontend page issues a fetch request or socket event based on user intent.
2. Express receives the request, validates required payload fields, and routes to feature logic.
3. Database-backed requests interact with PostgreSQL via the shared `db.js` pool.
4. Chatbot-backed requests are proxied to the local Python service using internal HTTP calls.
5. Responses return as JSON to the browser, where the UI updates local state and navigational behavior.
6. Realtime room events bypass REST and instead propagate over Socket.IO rooms.

5. Project Folder Structure

Folder / File	Purpose
<code>backend/server.js</code>	Main backend entry point, Express app setup, Socket.IO signaling, chatbot orchestration, feedback APIs, image upload API.
<code>backend/routes/auth.js</code>	Signup and login route handlers with bcrypt hashing and JWT token issuance.
<code>backend/routes/meetings.js</code>	Meeting creation and meeting lookup by code.
<code>backend/db.js</code>	Shared PostgreSQL connection wrapper using <code>pg</code> .
<code>backend/init.sql</code>	Schema bootstrap for users, meetings, participants, and chatbot learning events.

backend/chatbot/service.py	Custom chatbot engine, training, inference, image classification, and internal HTTP endpoints.
backend/chatbot/training_data.csv	Primary curated intent/answer/action dataset.
backend/chatbot/learning_feedback.csv	Incremental learned rows appended from user correction feedback.
backend/chatbot/image_training_data.csv	Image training dataset mapping image labels and image Q/A samples.
frontend/src/pages	User-facing pages: landing, auth, and meeting experience.
frontend/src/components/AIHelpBot.jsx	Cross-application chatbot widget, image upload, feedback capture, live state query initiation.
frontend/src/context/AuthContext.jsx	Simple global auth state using React context and localStorage.

6. Frontend Documentation

Frontend Pages

Page	Route	Responsibilities
Landing	/	Entry screen, new meeting creation, create-for-later flow, join-by-code flow, marketing showcase, auth navigation.

Login	/login	User sign-in form, API-driven credential submission, error display, token persistence through context.
Signup	/signup	User registration form, account creation, auto-login after successful signup.
Meeting Room	/room/:id	Video grid, WebRTC media lifecycle, chat, reactions, host controls, people panel, live chatbot meeting state bridge.

```

22  const MeetingRoom = () => {
482  useEffect(() => {
483    const answerLiveMeetingQuestion = (event) => {
501      const chatcount = messages.length;
502      const response = {
503        handled: false,
504        answer: '',
505      };
506
507      if (includesAny(normalizedQuestion, ['what is on the screen', 'what s on the screen', 'what can you see', 'what is visible', 'what
508        response.handled = true;
509        response.answer = 'You are on the meeting screen. The current visible area is the ${openPanel}. Your mic is ${micOnRef.current ?
510      } else if (includesAny(normalizedQuestion, ['is my mic on', 'is mic on', 'microphone status', 'mic status', 'am i muted', 'is my mi
511        response.handled = true;
512        response.answer = micOnRef.current ? 'Your microphone is currently on.' : 'Your microphone is currently off or muted.';
513      } else if (includesAny(normalizedQuestion, ['is my camera on', 'is camera on', 'camera status', 'video status', 'is my camera off',
514        response.handled = true;
515        response.answer = videoOnRef.current ? 'Your camera is currently on.' : 'Your camera is currently off.';
516      } else if (includesAny(normalizedQuestion, ['am i sharing screen', 'is screen sharing on', 'screen sharing status', 'am i presentin
517        response.handled = true;
518        response.answer = screenSharingRef.current ? 'Screen sharing is currently active.' : 'Screen sharing is not active right now.';
519      } else if (includesAny(normalizedQuestion, ['is chat open', 'chat panel open', 'which panel is open', 'what panel is open', 'is peo
520        response.handled = true;
521        response.answer = 'The currently open panel is ${openPanel}.';
522      } else if (includesAny(normalizedQuestion, ['how many participants', 'how many members', 'member count', 'participant count', 'how
523        response.handled = true;
524        response.answer = 'There ${totalMembers === 1 ? 'is' : 'are'} ${totalMembers} ${totalMembers === 1 ? 'participant' : 'participant
525      } else if (includesAny(normalizedQuestion, ['who is here', 'who is present', 'present members', 'participants list', 'who joined'])
526        response.handled = true;
527        response.answer = 'Currently present members: ${participantNames.join(', ')}.';
528      } else if (includesAny(normalizedQuestion, ['who am i', 'my name', 'what is my name'])) {
529        response.handled = true;
  
```

Fig : frontend files

Component and Routing Logic

- **App.jsx** wires the router and globally mounts the chatbot widget so it is available across routes.
- **main.jsx** bootstraps the React application in strict mode.
- **AuthContext.jsx** stores the authenticated user and JWT token in localStorage and exposes login / logout helpers.
- **AIHelpBot.jsx** manages chatbot conversation state, image analysis in the browser, image upload, feedback entry, and room-specific action triggers.

UI Logic and State Management

STATE MODEL

- Global auth state uses React Context only.
- Page-level interactions use component-local `useState` values.
- Meeting room state combines React state with refs to keep socket callbacks synchronized.
- No Redux, MobX, or external state container is present.

CLIENT-SIDE BEHAVIOR

- Landing page handles navigation, join-code input, and copy-link modal.
- Meeting page acquires media devices and creates peer connections dynamically.
- Chatbot widget performs image feature extraction before upload.
- Chatbot in a meeting room asks the current page for live-state answers before calling the backend.

Responsive Design and Visual Style

The UI uses a custom dark theme with glassmorphism surfaces, CSS variables, animation utilities, and flexible grids. The landing page contains explicit media-query handling, while the meeting room uses auto-fit grid layouts and floating panels to adapt to different room sizes. The visual language is consistent across auth, landing, meeting, and chatbot modules.

7.Backend Documentation

Server Architecture

The backend is centered in `backend/server.js`. It initializes Express, configures and JSON parsing, serves uploaded assets, mounts route modules, establishes Socket.IO on top of an HTTP server, initializes the PostgreSQL schema, and starts the Python chatbot process as an auxiliary local service.

BACKEND RESPONSIBILITIES

- Account creation and credential verification.
- Meeting creation and lookup by meeting code.
- Chatbot request orchestration and action resolution.
- Feedback persistence and retraining triggers.
- Image upload handling for chatbot image workflows.
- Realtime signaling, chat broadcast, reactions, hand toggles, and host control propagation.

Routes and Business Logic

Module	Key Logic
<code>routes/auth.js</code>	Checks for duplicate usernames, hashes passwords with bcrypt, issues JWT tokens on signup/login.
<code>routes/meetings.js</code>	Generates meeting codes, creates meeting records, retrieves meeting details by code.
<code>server.js</code> <code>/api/chatbot/*</code>	Forwards requests to the Python chatbot, resolves supported actions such as meeting creation and user creation, stores learning events, uploads chatbot images, triggers retraining.

Realtime Layer (Socket.IO)

Event	Direction	Purpose
<code>join-room</code>	Client → Server	Join a room, register host if first user, and initialize room config.
<code>room-config</code>	Server → Client	Sends host status and moderation settings to joining user.
<code>all-users</code>	Server → Client	Returns existing room users to the newly joined client.
<code>signal</code>	Bi-directional	Exchanges WebRTC offer/answer/ICE candidate data.
<code>send-chat-message / chat-message</code>	Bi-directional	Broadcasts text chat to room participants.

<code>send-reaction / receive-reaction</code>	Bi-directional	Broadcasts emoji reactions.
<code>toggle-hand / hand-toggled</code>	Bi-directional	Updates raised-hand state for participants.
<code>update-host-controls / host-controls-updated</code>	Bi-directional	Synchronizes host moderation settings.
<code>user-disconnected</code>	Server → Client	Notifies peers to remove a departed user.

8.Database Design

The relational schema is declared in `backend/init.sql` and currently contains four primary tables. The schema is minimal but functional, supporting identity, meeting persistence, participant audit history, and chatbot correction history.

Table	Purpose	Important Fields
<code>users</code>	Stores registered user accounts.	<code>id</code> , <code>username</code> , <code>password</code> , <code>created_at</code>
<code>meetings</code>	Stores meeting records and host linkage.	<code>id</code> , <code>meeting_code</code> , <code>host_id</code> , <code>created_at</code> , <code>is_active</code>
<code>meeting_participants</code>	Tracks participant join/leave timestamps per meeting.	<code>meeting_id</code> , <code>user_id</code> , <code>joined_at</code> , <code>left_at</code>
<code>chatbot_learning_events</code>	Stores chatbot feedback events, rewards, predicted/corrected intent, and correction history.	<code>question</code> , <code>bot_answer</code> , <code>predicted_intent</code> , <code>reward</code> , <code>corrected_answer</code> , <code>updated_at</code>

Relationships

- `meetings.host_id` references `users.id` with ON DELETE SET NULL.
- `meeting_participants.meeting_id` references `meetings.id` with ON DELETE CASCADE.
- `meeting_participants.user_id` references `users.id` with ON DELETE CASCADE.
- `chatbot_learning_events.user_id` references `users.id` with ON DELETE SET NULL.

Non-Relational Supporting Data

CSV / JSON DATA ASSETS USED BY THE PROJECT

- `training_data.csv` and `learning_feedback.csv` store chatbot training rows.
- `image_training_data.csv` stores image labels, image paths, dimensions, and image Q/A samples.
- `model.json` and `image_model.json` persist trained classifier output.

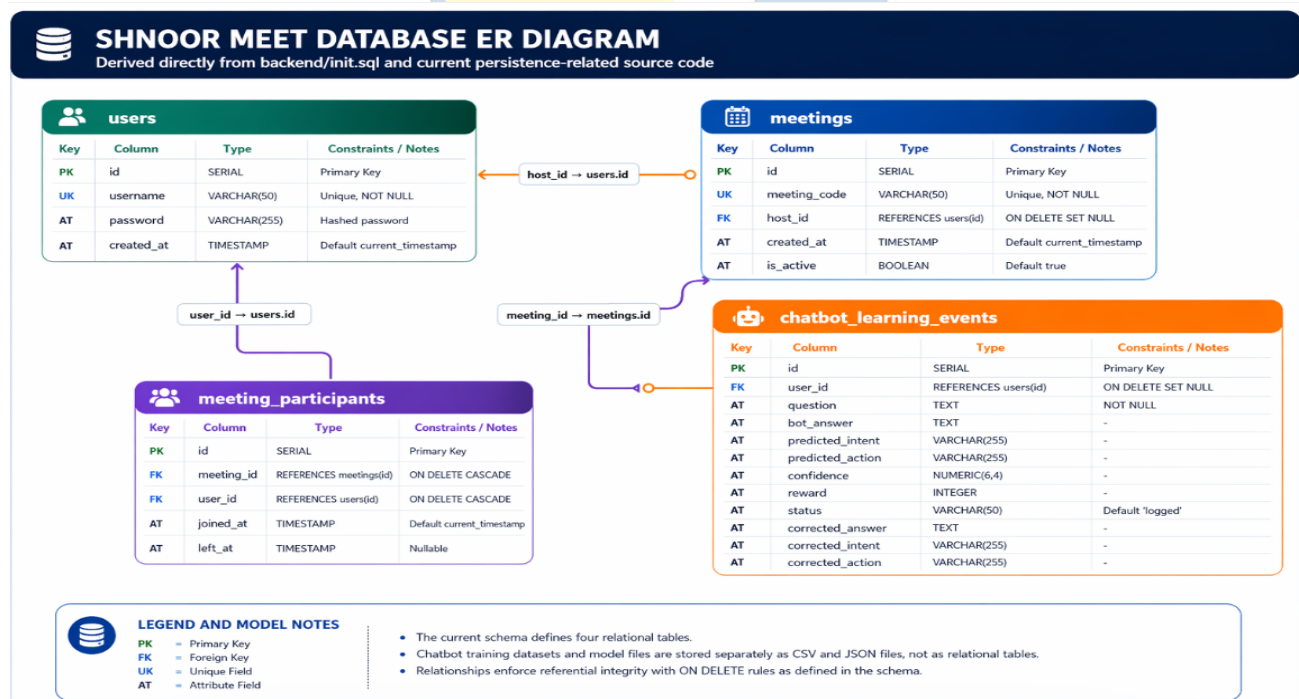


Fig : ER Diagram for Database Design

9. API Documentation

Public Backend APIs

Method	Endpoint	Purpose	Auth Required
POST	/api/auth/signup	Create a user account, hash password, return JWT and user payload.	No
POST	/api/auth/login	Validate credentials and return JWT plus user summary.	No
POST	/api/meetings/create	Create a new meeting record and generate a meeting code.	No enforced middleware
GET	/api/meetings/:code	Fetch a meeting record by meeting code.	No enforced middleware
POST	/api/chatbot/ask	Request chatbot answer, optional image context, and action resolution.	No enforced middleware
POST	/api/chatbot/feedback	Save positive/negative chatbot feedback and retrain the model when corrected.	No enforced middleware
POST	/api/chatbot/upload-image	Upload a chatbot image attachment and return a served URL.	No enforced middleware
POST	/api/chatbot/train	Force chatbot retraining from CSV datasets.	No enforced middleware

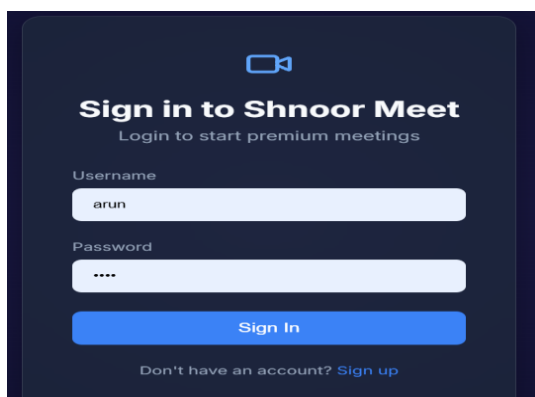
Internal Python Chatbot Service APIs

Method	Endpoint	Purpose	Exposure
GET	/health	Health check and basic model statistics.	Internal local service
POST	/train	Retrain the text chatbot model.	Internal local service
POST	/ask	Run chatbot inference with text and optional image context.	Internal local service
POST	/learn-image	Append an image sample and retrain the image classifier.	Internal local service

10. Modules & Features

10.1 Authentication & User Access Module

- User signup and login forms connected to backend auth endpoints.
- JWT issuance and client-side persistence via localStorage.
- Logout behavior through AuthContext state reset.
- Chatbot-triggered user creation supported via natural-language command parsing.



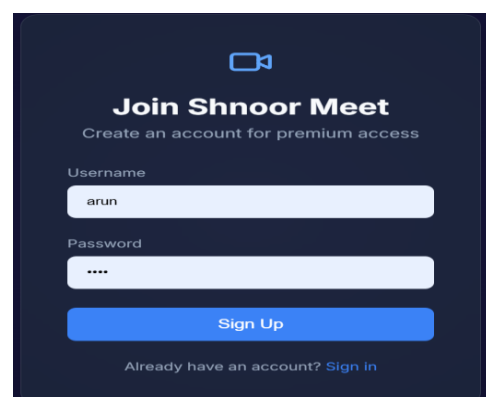
Sign in to Shnoor Meet
Login to start premium meetings

Username
arun

Password
....

Sign In

Don't have an account? [Sign up](#)



Join Shnoor Meet
Create an account for premium access

Username
arun

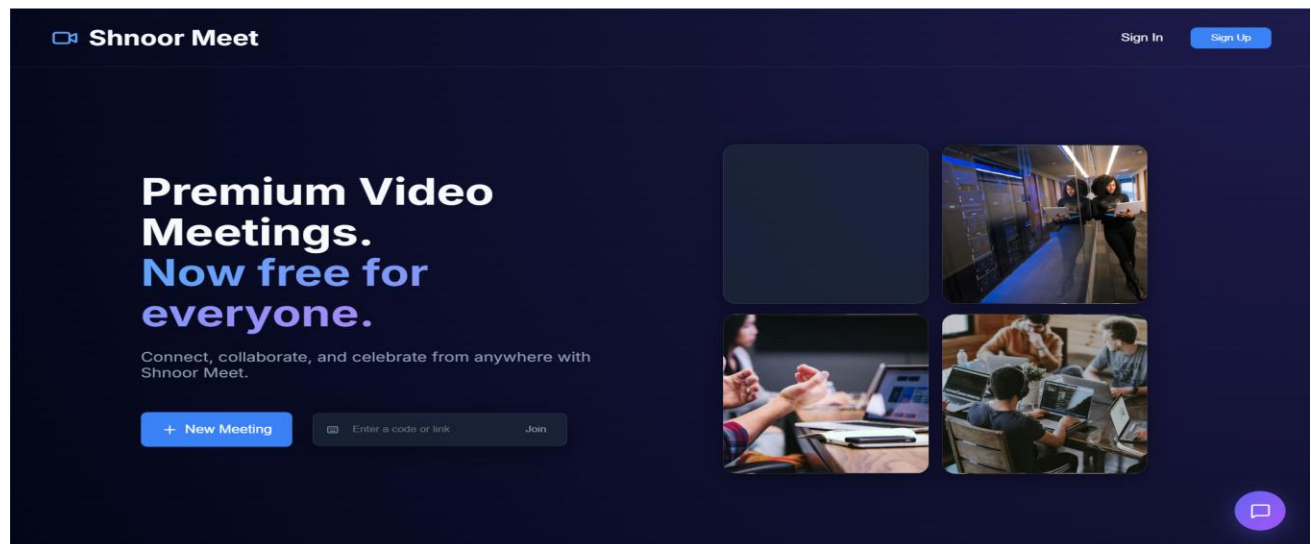
Password
....

Sign Up

Already have an account? [Sign in](#)

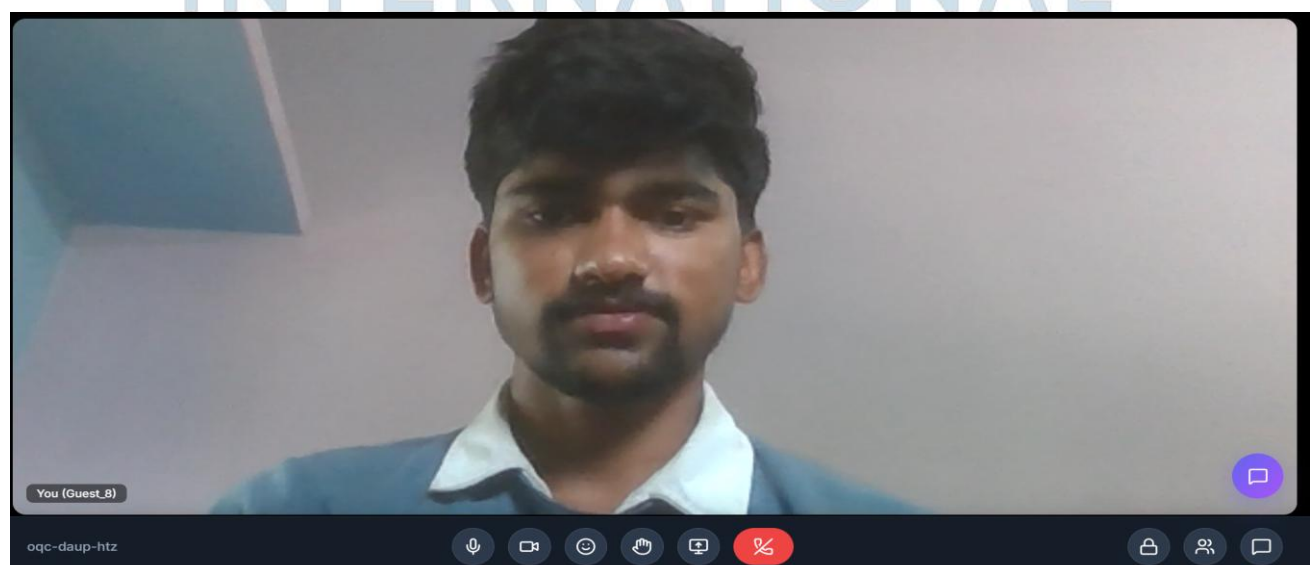
10.2 Landing & Meeting Launch Module

- Create an instant meeting and navigate directly to the room.
- Create a meeting for later and copy a generated join link.
- Join an existing meeting by entering a meeting code or link-like value.
- Display signed-in user context and auth navigation actions.



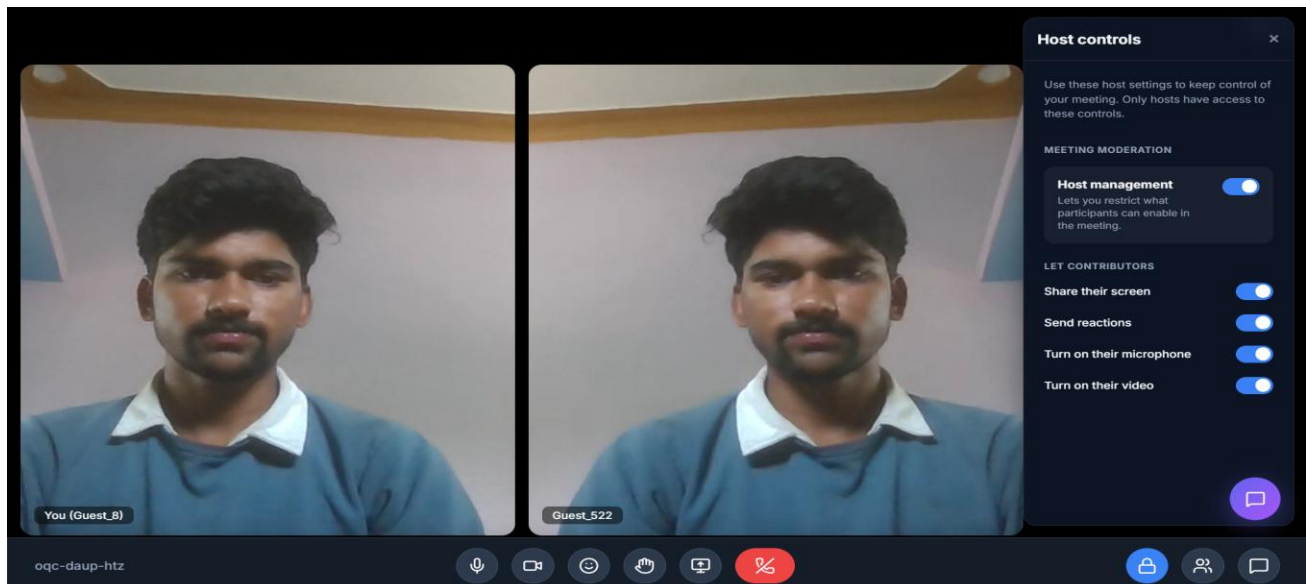
10.3 Meeting Room & Collaboration Module

- Acquires camera and microphone media using browser media APIs.
- Builds peer connections dynamically for every participant through Socket.IO signaling.
- Supports chat messages, reactions, hand raise, and floating emoji display.
- Supports screen sharing through track replacement on RTCPeerConnection senders.



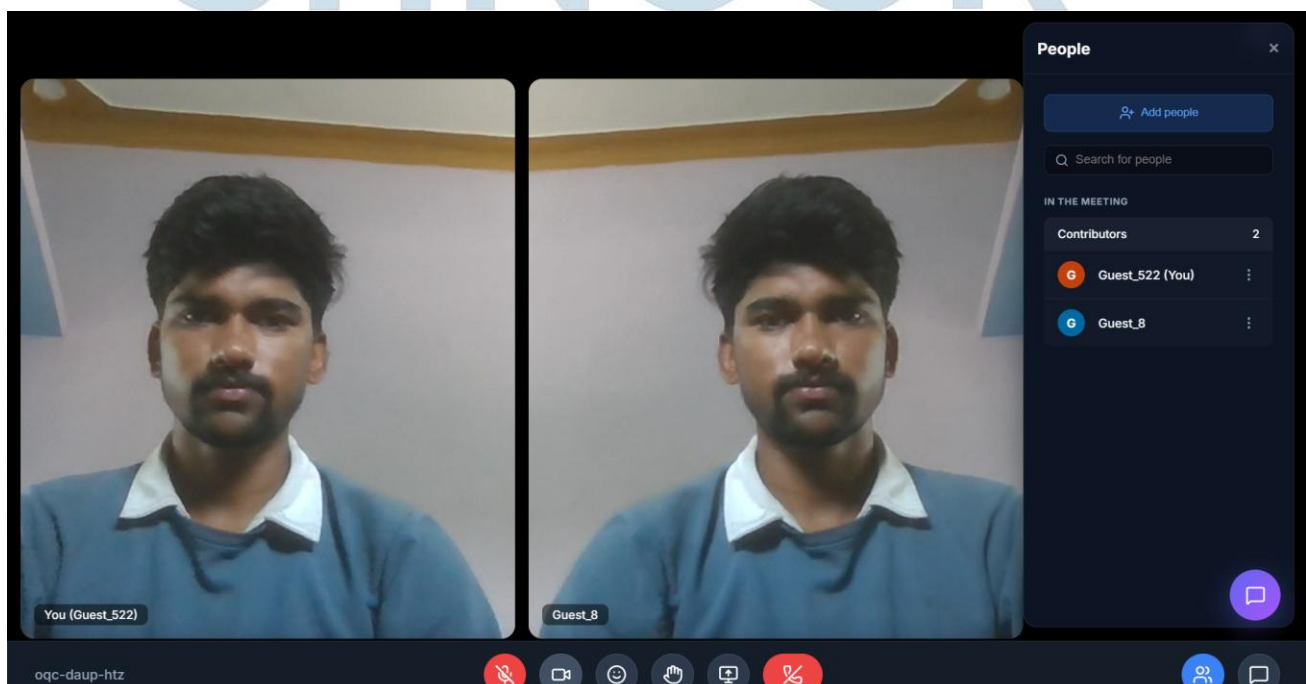
10.4 Host Controls & Moderation Module

- The first user in a room is recognized as host.
- Host can enable moderation mode and restrict screen share, reactions, microphone activation, and video activation.
- Restrictions are propagated to all participants over Socket.IO.



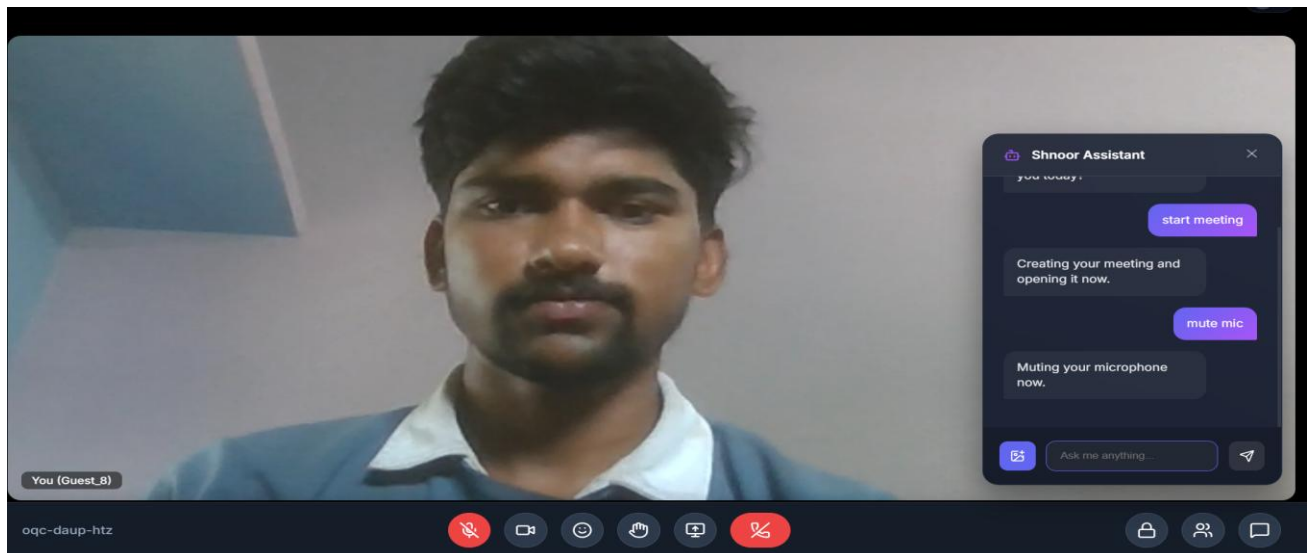
10.5 People & Chat Module

- People panel lists current participants and identifies the local user / host.
- Chat panel shows sender names, timestamps, and room message history.
- Meeting link copy helper is available from the people panel.



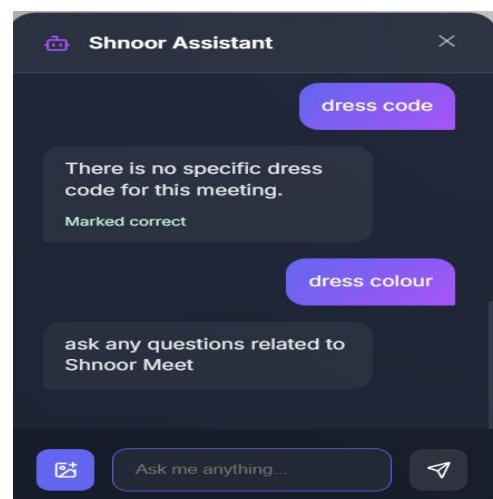
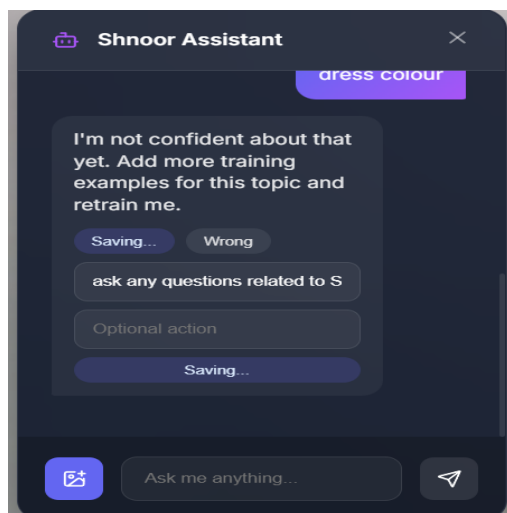
10.6 AI Chatbot Module

- Cross-route chatbot widget with conversation history.
- Intent classification trained from `training_data.csv` and `learning_feedback.csv`.
- Action execution for meeting creation, meeting link generation, joining, leaving, in-room controls, and user creation.
- Internal Python service hosted separately and called from Express.



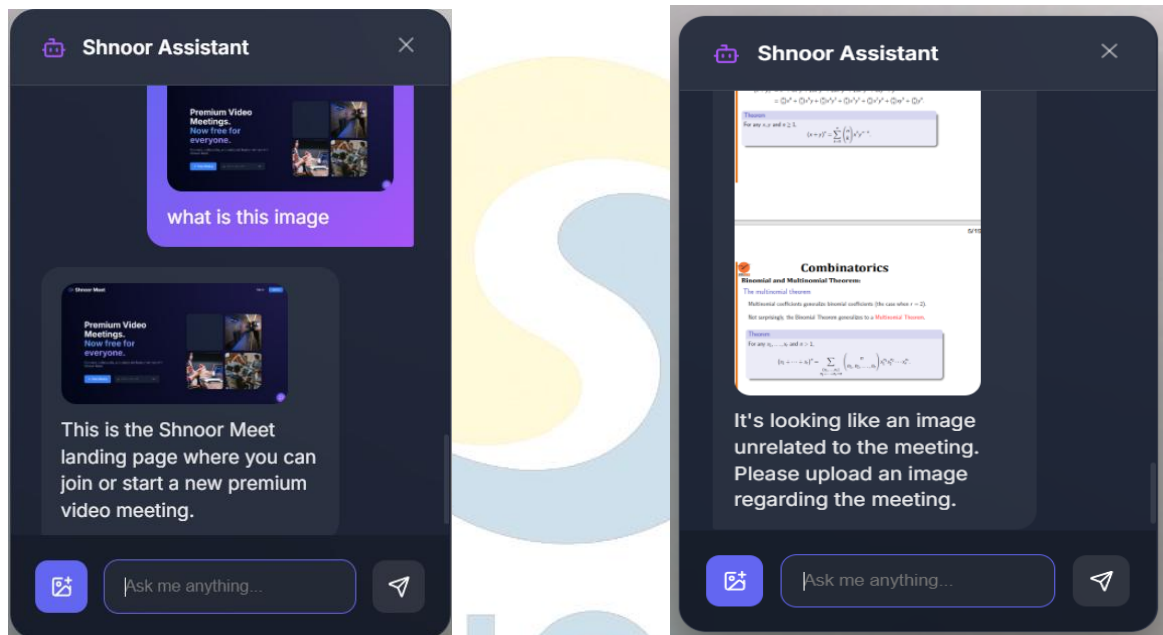
14.7 Feedback Learning Module

- Low-confidence chatbot answers are marked for review and expose Correct / Wrong feedback actions.
- Wrong answers can be corrected in-app and appended to the learned CSV dataset.
- Each event is logged to PostgreSQL in `chatbot_learning_events`.
- Retraining is triggered so the same question can be answered correctly next time.



14.8 Image-Based Learning Module

- Users can attach an image in the chatbot.
- The browser computes image features such as hash, dimensions, brightness, dominant color, and orientation.
- Images are uploaded to the backend and linked into image training data.
- Image Q/A samples can be corrected and retrained, allowing future similar-image answers.



15. Workflow Explanation

End-to-End User Flow

1. User lands on the home page and either signs in, signs up, or proceeds as a guest-like room participant.
2. User creates a new meeting or joins an existing one using a code.
3. Meeting page acquires media devices, opens a Socket.IO connection, and joins the selected room.
4. Socket signaling exchanges WebRTC offers, answers, and ICE candidates to establish media sessions.
5. During the meeting, participants can use chat, reactions, hand raise, and screen share.
6. If the room creator is present, host controls can govern participant capability toggles.

7. Chatbot requests are handled via live room state first, then backend text/image inference if no live answer applies.
8. Incorrect chatbot answers can be corrected, logged, written to training data, and retrained for future accuracy.

21. Conclusion

Shnoor Meet is a thoughtfully integrated collaboration application rather than a simple video-room demo. The codebase demonstrates full-stack coordination across React, Express, PostgreSQL, Socket.IO, WebRTC, and a locally trained Python chatbot engine. Its strongest differentiator is the assistant layer: the project not only embeds a help bot, but also supports CSV-driven training, image-based learning, actionable intents, correction-based retraining, and live meeting-state responses.

From an engineering perspective, the project is already strong as a functional prototype or academic/portfolio-grade product. From a production perspective, the next phase should focus on security hardening, automated testing, deployment formalization, and operational reliability. Even in its current form, the source code reflects a clear product direction, practical architectural separation, and meaningful innovation through custom chatbot integration.

The logo for SHNOOR INTERNATIONAL features a large, stylized 'S' in the background, composed of a yellow circle and a blue circle. The word 'SHNOOR' is written in large, light blue, sans-serif capital letters, and the word 'INTERNATIONAL' is written in smaller, light blue, sans-serif capital letters below it.

SHNOOR
INTERNATIONAL